Available online at www.sciencedirect.com
SCIENCE DIRECT®
Computers & Operations Research 34 (2007) 28–47
computers & operations research
www.elsevier.com/locate/cor

Adaptive memory programming for the vehicle routing problem with multiple trips

Alfredo Olivera*, Omar Viera

Instituto de Computación, Facultad de Ingeniería, Universidad de la República, Herrera y Reissig 525, Montevideo, Uruguay

Available online 3 June 2005

Abstract

The Vehicle Routing Problem with Multiple Trips is an extension of the classical Vehicle Routing Problem in which each vehicle may perform several routes in the same planning period. In this paper, an adaptive memory algorithm to solve this problem is proposed. Computational experience is reported over a set of benchmark problem instances.
© 2005 Elsevier Ltd. All rights reserved.

Keywords: Vehicle routing; Multiple trips; Adaptive memory procedure

1. Introduction

The problem of distributing goods from depots to final consumers plays an important role in the management of many distribution systems, and its adequate programming may produce significant savings. The *Vehicle Routing Problem* (VRP) and its many variations have been subject of research during the last four decades. Some well studied characteristics include the existence of demands, time windows and heterogeneous vehicles [1].

However, some aspects that arise in real applications have not received much attention in the Operations Research literature. For instance, it usually assumed that each vehicle may perform at most

* Corresponding author.
E-mail addresses: aolivera@fing.edu.uy (A. Olivera), viera@fing.edu.uy (O. Viera).
URL: http://www.fing.edu.uy/∼aolivera (A. Olivera).

0305-0548/$ - see front matter © 2005 Elsevier Ltd. All rights reserved.
doi:10.1016/j.cor.2005.02.044

one route in the same planning period, and in many cases the number of available vehicles is supposed to be unlimited. In many practical applications, this assumptions are unrealistic. When the vehicle capacity is small or when the planning period is large, performing more than one route per vehicle may be the only practical solution. In urban areas, where travel times are rather small, it is often the case that after performing short tours vehicles are reloaded and used again.

The *Vehicle Routing Problem with Multiple Trips* (VRPMT) overcomes the mentioned limitations, besides considering the classic VRP constraints. Solving the VRPMT not only implies the design of a set of routes, but the assignment of those routes to the available vehicles. This makes the VRPMT a very practical problem, specially at an operational level, in which daily driver schedules must be designed for a fixed vehicle fleet.

In this paper we describe a heuristic to solve the VRPMT, which is based on the Adaptive Memory Procedure (AMP) proposed by Rochat and Taillard [2]. A definition of the VRPMT is given in Section 2, as well as a literature review. The proposed algorithm is described in Section 3. Computational behavior of the algorithm is reported and analyzed in Section 4, while conclusions and future work are considered in Section 5.

2. The VRP with multiple trips

Let $G = (V, E)$ be a graph where $V = \{0, 1, \ldots, n\}$ is the set of nodes and $E \subseteq V \times V$ is the set of arcs. If $(i, j) \in E$, then it is possible to travel from i to j, incurring in a cost c_{ij} and a travel time t_{ij}. Node 0 represents a depot where a fleet $K = \{1, \ldots, m\}$ of identical vehicles is based. Each vehicle has a limited capacity Q. The nodes in $V \backslash \{0\}$ represent customers, each one having a demand q_i. Finally, there exists a time horizon, denoted by T, which establishes the duration of a working day. It is assumed that Q, q_i and T are nonnegative integers.

The VRPMT calls for the determination of a set of routes and an assignment of each route to one vehicle, which minimizes the total routing costs and satisfies the following conditions:

(1) each route starts and ends at the depot,
(2) each customer is visited by exactly one route,
(3) the demand of the customers in the same route does not exceed Q,
(4) the duration of routes assigned to the same vehicle does not exceed T.

The VRPMT is a generalization of the VRP: any VRP instance can be transformed to an equivalent VRPMT instance, setting $m = n$ and $T = \sum_{(i,j)\in E} t_{ij}$. As the VRP is an NP-Hard problem [3], the VRPMT is also NP-Hard. The computational complexity of the VRPMT justifies the use of heuristics to solve instances of realistic size in moderate computation times.

2.1. Literature review

Fleischmann [4] was first one to address the problem. He proposed a modification to the savings algorithm [5] and used a Bin Packing Problem (BPP) [6] heuristic to assign the routes to the vehicles.

Taillard et al. [7] proposed a three phase algorithm to solve the problem. In the first phase, a set of routes satisfying capacity constraints is constructed. Next, those routes are combined into complete VRP solutions. Finally, the routes of each solution are assigned to the vehicles solving a BPP.

Brandão and Mercer [8] designed a tabu search algorithm to solve a real life application which included, among others, the VRPMT features. Later, they simplified the algorithm [9] to handle the VRPMT. Key features are shared by both proposals. Moves are defined by swapping two customers and by removing one customer from its route and inserting it into another one. Insertions are made using the GENI algorithm [10]. Infeasible solutions are visited during the search and penalized using a modified objective function.

Petch and Salhi [11] proposed a multi-phase heuristic which first constructs many feasible VRP solutions and, for each one, assigns routes to vehicles using a BPP heuristic. Feasible VRP solutions are built with repeated executions of Yellow's savings algorithm [12] and, independently, using a route population approach.

2.2. Notation

A route will be denoted as a sequence of nodes $r = (v_0, \ldots, v_{n_r+1})$, where $v_0 = v_{n_r+1} = 0$ and $(v_i, v_{i+1}) \in E \; \forall i \in [0, n_r]$. For each route r, n_r is the number of customers it visits, $q_r = \sum_{i=1}^{n_r} q_{v_i}$ is the demand covered by the route, $c_r = \sum_{i=0}^{n_r} c_{v_i, v_{i+1}}$ is the cost and $t_r = \sum_{i=0}^{n_r} t_{v_i, v_{i+1}}$ is the duration. A route is feasible if $q_r \leqslant Q$.

A solution is a sequence of route sets $s = (R_1, \ldots, R_m)$, where R_k contains the routes assigned to vehicle k. The set of all the routes in a solution is $R(s) = \bigcup_{k \in K} R_k$. The cost of a solution is

$$f(s) = \sum_{r \in R(s)} c_r. \tag{1}$$

The total duration of routes assigned to each vehicle cannot be greater than the time horizon (these will be called *overtime constraints*). A measure of such constraint violation, called *overtime*, is defined for each vehicle:

$$O_k(s) = \left(\sum_{r \in R_k} t_r - T \right)^+, \tag{2}$$

where $a^+ = \max(a, 0)$. Finally, the *total overtime* of a solution is given by

$$O(s) = \sum_{k \in K} O_k(s). \tag{3}$$

2.3. Integer programming formulation

Let R be a set containing all feasible routes. For each $r \in R$ define a parameter a_{ir} indicating whether route r visits customer i ($a_{ir} = 1$) or not ($a_{ir} = 0$). The VRPMT can be formulated as an 0-1 Linear

Program, as follows:

$$\min \sum_{k\in K}\sum_{r\in R} c_r x_r^k \tag{4}$$

$$\text{s.t.} \sum_{k\in K}\sum_{r\in R} a_{ir} x_r^k = 1, \quad \forall i \in V\backslash\{0\}, \tag{5}$$

$$\sum_{r\in R} t_r x_r^k \leqslant T, \quad \forall k \in K, \tag{6}$$

$$x_r^k \in \{0, 1\}, \quad \forall k \in K,\ \forall r \in R.$$

This formulation resembles the Set Partitioning based VRP formulation due to Balinsky and Quandt [13]. If $x_r^k = 1$, then r is part of the solution and it is assigned to vehicle k (i.e. $r \in R_k$). If $x_r^k = 0\,\forall\, k \in K$, r is not part of the solution. The objective function (4) establishes that the total routing cost should be minimized. The fact that each customer must belong to exactly one route is imposed in (5) and overtime constraints are established in (6).

Despite having an exponential number of variables, this formulation emphasizes the structure of VRPMT constraints. The bounded capacity of vehicles and the fact that routes must start and end at the depot are *local* to each route, and thus, can be encapsulated in the definition of R. Visiting each customer once and having no overtime are *global*, as they affect the whole solution. Violations to local constraints can be detected examining each route separately, but to ensure global constraints' satisfaction, the whole solution needs to be analyzed.

The presence of a fixed number of vehicles makes the overtime constraints hard to satisfy in the general case. On one hand, as there are only m vehicles available, the number of routes per vehicle tends to increase when m decreases. On the other hand, T imposes an upper bound on the total duration of routes assigned to each vehicle. For tight problems, feasible solutions can be hard to find and may not exist.

2.4. Assigning routes to vehicles

The problem of assigning routes to the vehicles arises repeatedly when solving the VRPMT [4,7,11] and can be solved by means of a BPP [6]. Given a set of routes $\{r_1, \ldots, r_p\}$, a feasible assignment of those routes to the available vehicles exists if p items of weights $t_{r_1}, \ldots, t_{r_p}$ can be packed in m (or less) bins of capacity T.

However, a two phase approach that solves a VRP and assigns the obtained routes via a BPP may lead to infeasible solutions, as it is shown in the following example. Consider the VRP benchmark problem of 50 customers CMT-1, proposed by Christofides et al. [14]. An optimal solution to that instance is shown in Fig. 1a. It is composed of five routes of durations $t_1 = 98.45$, $t_2 = 99.25$, $t_3 = 99.33$, $t_4 = 109.06$ and $t_5 = 118.52$ and has a total cost of 524.61. Consider a VRPMT instance over the same nodes, with the same vehicle capacity, $m = 4$ and $T = 144$. No feasible solution to this new instance can be constructed keeping the routes of the given VRP solution, because no packing of items of weights $t_1, \ldots, t_5$ in four bins of capacity 144 exists. However, a feasible solution to this instance exists, and an example is given in Fig. 1b. Its total cost is 546.29 and it has five routes of durations 57.82, 82.12, 122.80, 141.76 and 141.79, the first two being assigned to the same vehicle and the other three assigned each to a different one.

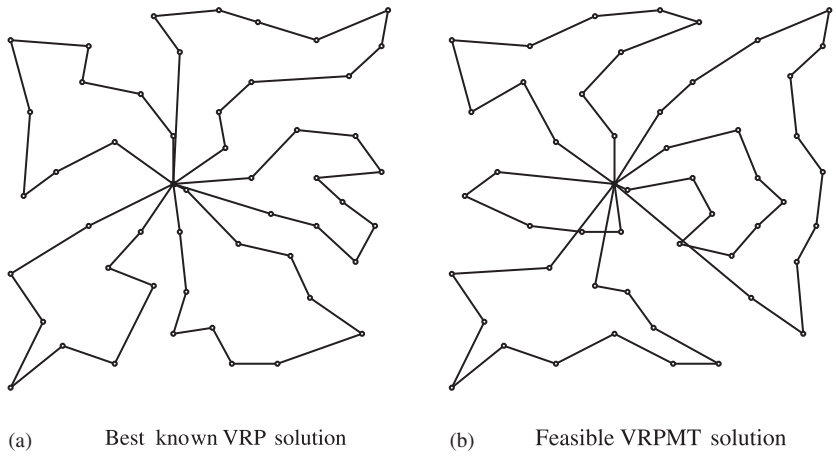

Fig. 1. Solutions for VRP and VRPMT formulations over CMT-1 problem: (a) Best known VRP solution, (b) Feasible VRPMT solution.

3. Adaptive memory algorithm

The AMP was first proposed by Rochat and Taillard [2] as an enhancement of Tabu Search (TS) to solve the VRP. It was motivated by the work of Glover regarding surrogate constraints [15]. An important principle behind AMP is that good solutions may be constructed by combining different *components* of other good solutions. A memory containing components of visited solutions is kept. Periodically, a new solution is constructed using the data in the memory and improved by a local search procedure. The improved solution is then used to update the memory. A sketch of the AMP is given below:

(1) Initialize the memory M.
(2) While a stopping criteria is not met, do:
  (a) Construct a new solution s combining components of M.
  (b) Apply a local search procedure to s (let s^* be the improved solution).
  (c) Update M using components of s^*.

In the first iterations, the memory is expected to contain components of many different solutions. Thus, solutions constructed in Step 2a will differ substantially among iterations and diverse regions of the search space will be explored by the local search. As the whole procedure evolves, modifications made by the local search will be minor, making the components in the memory belong to a small set of good solutions. Thus, the search is intensified over promising regions.

Rochat and Taillard [2] used the AMP to solve the VRP and the VRP with Time Windows. In their proposal, single vehicle routes are stored in the adaptive memory and new solutions are constructed by combining those routes. In the algorithms proposed by Tarantilis and Kiranoudis [16] and Tarantilis [17] to solve the VRP, new solutions are built by combining certain sequences of consecutive customers. The Heterogeneous Fleet VRP [18] and the Multiple Depot VRP with Inter-Depot routes [19] have also been tackled with the AMP methodology.

Other problems, such as boolean optimization problems [20], political districting [21] and team orienteering [22], have also been tackled with the AMP methodology. Ahmadi and Osman [23] proposed an hybridization between GRASP and AMP to solve the Capacitated Clustering Problem.

As it was noticed by Taillard et al. [24], the AMP may be viewed as a generalization of many well known meta-heuristics such as Genetic Algorithms, Simulated Annealing, Tabu Search, Scatter Search and Ant Systems.

3.1. The algorithm

In our proposal, the memory is a sorted multi-set of routes. The AMP may be viewed as an attempt to generate a promising subset M of the set R defined in Section 2.3. At every iteration, a new solution is constructed non-deterministically by selecting some of the routes in M. The solution is improved using a TS procedure, and the routes are added to M. Infeasible solutions may be visited during the execution. In the TS, solutions may violate capacity and overtime constraints. In the rest of the AMP only overtime constraints may be violated.

The approach is very similar to that of Rochat and Taillard [2], being three the main differences. First, when a solution must be built from scratch, Rochat and Taillard [2] generate single customer routes and apply the local search, while in our case the Sweep algorithm [25] is used. Second, they use the TS proposed by Taillard [26] and we use a modification of the Taburoute [27] algorithm. Finally, in the original approach routes are labelled with the cost of the solution to which they belong. As we accept infeasible solutions, two dimensional labels are used to account for infeasibility and cost.

The main cycle is ran for AMP^{iter} iterations. In the rest of this section, each of the AMP components is explained: memory initialization, memory update, solution construction and TS procedure.

3.1.1. Memory initialization

To initialize the memory, I solutions are generated and inserted into M. Each solution is built running one iteration of the Sweep algorithm [25] starting from a randomly selected customer. The TS algorithm explained in Section 3.1.4 is used to improve each solution before inserting it in the memory.

3.1.2. Memory update

The memory is stored as a sorted list of routes. Each route is labelled with $(O(s), f(s))$ being s the solution to which it belongs. The lexicographic criteria is used for sorting, as it ensures that routes that are part of good solutions occupy the first positions. Let r_1 and r_2 be two routes belonging to solutions s_1 and s_2 respectively. If r_1 precedes r_2 in M, then one of the following conditions holds:

- $O(s_1) < O(s_2)$, which means that s_1 is nearer to feasibility than s_2, or
- $O(s_1) = O(s_2)$ and $f(s_1) \leqslant f(s_2)$, which means that both are equivalent in terms of feasibility but s_2 is not better in terms of cost.

When a solution is added to the adaptive memory, each of its routes is labelled and inserted preserving the list ordering. If $|M|$ is too high, the probability of selecting good routes (see Section 3.1.3) may be diminished by the routes in the last positions. The parameter M^{size} specifies the maximum size of M. Whenever $|M| > M^{\text{size}}$, the last $|M| - M^{\text{size}}$ routes are removed from the memory. This bounding also contributes to space and time optimization of the AMP.

3.1.3. Solution construction

New solutions are built by probabilistically selecting routes from the memory. Higher probability is assigned to the routes in the first positions, because they belong to the best solutions found so far. The procedure is specified next:

(1) Set $M' \leftarrow M$.
(2) Probabilistically select $r \in M'$.
(3) Remove from M' all the routes having customers in common with r.
(4) If all customers have been routed, stop. If $M' \neq \emptyset$, go to Step 2, else go to Step 5.
(5) Solve a VRP with the unrouted customers and the depot using the Sweep algorithm (starting from a random customer) and add the routes to the solution.

The probability of selecting the i-th route of M' is $2\frac{|M'|+1-i}{|M'|(|M'|+1)}$.

3.1.4. Tabu search procedure

A TS procedure is used to improve the constructed solution. Violations to capacity and overtime constraints are allowed, but penalized using the following objective function:

$$f_P(s) = f(s) + \alpha \sum_{r \in R(s)} (q_r - Q)^+ + \beta O(s). \tag{7}$$

The parameter α penalizes the total capacity constraints violation and β penalizes the total overtime. The values of α and β are updated at every iteration, as it is done in the Taburoute algorithm for the VRP [27], producing an oscillation between feasible and infeasible solutions.

Given a solution s, its neighborhood $N(s)$ is defined as the set of solutions that can be obtained applying one of the following operations to any pair of different routes $r_i = (\ldots, v_{i-1}, v_i, v_{i+1}, \ldots)$ and $r_j = (\ldots, w_{j-1}, w_j, w_{j+1}, \ldots)$ of $R(s)$:

Swap: set $r_i \leftarrow (\ldots, v_{i-1}, w_j, v_{i+1}, \ldots)$ and $r_j \leftarrow (\ldots, w_{j-1}, v_i, w_{j+1}, \ldots)$.
Insert: set $r_i \leftarrow (\ldots, v_{i-1}, v_{i+1}, \ldots)$ and $r_j \leftarrow (\ldots, w_{j-1}, v_i, w_j, w_{j+1}, \ldots)$.

In order to reduce the size of $N(s)$, and the running time of the search, a move is examined only if v_i is one of the p closest nodes to v_j and vice-versa. This reduced neighborhood will be called $N_p(s)$.

During the search, some moves are declared to be tabu and cannot be applied until its tabu status is revoked. After removing v from r, inserting it again in the same route is declared tabu for a random number of iterations. As usual, the tabu status of a move can be overridden if its application improves the best solution found so far (this is called *aspiration criteria*).

Let $N_p^*(s)$ be $N_p(s)$ restricted to moves that are not tabu or pass the aspiration criteria. Being s the incumbent solution, the whole $N_p^*(s)$ must be examined, and one solution minimizing f_P is set as the new incumbent solution.

After applying a move, a new assignment of routes to vehicles is computed by the heuristic given in Section 3.2. This is a major departure from previous approaches to the VRPMT, which make the assignment at the beginning [9] or at the final stage [7,11], but not at every iteration.

Whenever a new best solution is obtained, each of the routes that were modified by the last move is rearranged by means of the Unstringing and Stringing (US) post-optimization procedure [10]. The

parameter p of the US procedure was set to 5, which gives a good compromise between running time and solution quality [10]. The TS starts from a feasible VRP solution s and proceeds as follows:

(1) Assign the routes of s applying the heuristic given in Section 3.2.
(2) Initialize the tabu list. Set $\alpha \leftarrow \alpha^{\min}$, $\beta \leftarrow \beta^{\min}$ and $s^{\text{best}} \leftarrow s$.
(3) Repeat TS^{iter} times
  (a) Find $s' \in N_p^*(s)$ minimizing f_P.
  (b) If v was removed from r, inserting v in r is declared tabu for the next θ iterations, where θ is drawn uniformly in $[\theta^{\min}, \theta^{\max}]$.
  (c) Assign the routes of s' applying the heuristic given in Section 3.2.
  (d) If s' is better than s^{best}, apply the US procedure to the every route in s' and set $s^{\text{best}} \leftarrow s'$.
  (e) Update α and β.
  (f) Set $s \leftarrow s'$.
(4) Returns^{best}.

In Step 3d, s' is better than s^{best} if the routes of s' satisfy the capacity constraints and one of the following conditions hold:

- s' is "less infeasible" than s^{best}: $O(s') < O(s^{\text{best}})$.
- Both are equivalent in terms of feasibility, but s' has a smaller cost: $O(s')=O(s^{\text{best}})$ and $f(s')<f(s^{\text{best}})$.

The parameter update of Step 3e is done as follows. If s' satisfies the capacity constraints, then α is divided by 2, if not, it is multiplied by 2. The same is done with β, but with respect to overtime constraints. Lower and upper bounds $\alpha^{\min}$, $\beta^{\min}$, $\alpha^{\max}$ and $\beta^{\max}$ are imposed to α and β.

3.2. Assignment heuristic

During the TS, a VRPMT solution $s = (R_1, \ldots, R_m)$ must be constructed from a set of routes R'. The following heuristic is used:

(1) Initialize the assignment: $R_k \leftarrow \emptyset \; \forall k \in K$.
(2) Select the longest route that has not been assigned: $r^* \leftarrow \arg\max_{r \in R'} t_r$.
(3) Select the emptiest vehicle: $k^* \leftarrow \arg\min_{k \in K} \sum_{r \in R_k} t_r$.
(4) Assign r^* to k^*: $R_k \leftarrow R_k \cup \{r^*\}$.
(5) Disregard r^*: $R' \leftarrow R' \backslash \{r^*\}$.
(6) If $R' = \emptyset$, stop, else, go to Step 2.

The obtained solution is not guaranteed to satisfy the overtime constraints. When the resulting assignment is infeasible the following procedure is run, trying to obtain a feasible solution:

(1) Select the vehicle k^* which finishes later: $k^* = \arg\max_{k \in K} \sum_{r \in R_k} t_r$.
(2) If all routes in R_{k^*} have been marked, stop. Select an unmarked $r^* \in R_{k^*}$. Mark r^*.
(3) If there exists $k' \in K$ and $r' \in R_{k'}$ such that:

$O_{k'} = 0$, $t_{r'} < t_{r^*}$ and $\sum_{r \in R_{k'}} t_r - t_{r'} + t_{r^*} \leqslant T$
go to Step 4. If not, go to Step 2.

(4) Set $R_{k^*} \leftarrow R_{k^*} \cup \{r'\} \backslash \{r^*\}$ and $R_{k'} \leftarrow R_{k'} \cup \{r^*\} \backslash \{r'\}$. If the solution is feasible, stop. If not, go to Step 1.

This algorithm tries to reduce the duration associated to the vehicle that arrives later, by exchanging a route with another vehicle. After routes are swapped in Step 4, vehicle k^* finishes earlier than before and k' finishes later (but within the time horizon). Again, feasibility is not guaranteed at this stage.

4. Computational experiments

4.1. Benchmark problems

The algorithm was tested over the 104 benchmark problems proposed by Taillard et al. [7]. These were constructed using the same graphs, demands and vehicle capacities than the problems 1–5 and 11, 12 proposed by Christofides et al. [14] and problems 11 and 12 proposed by Fisher [28] for the VRP. For each VRP instance, several values of m are used. For each m, two time horizons $T_1 = [1.05\, z^*/m]$ and $T_2 = [1.10\, z^*/m]$ are proposed, where z^* is the best known solution value for the VRP instance and $[a]$ denotes a rounded to the nearest integer. All the problems are Euclidean and satisfy $t_{ij} = c_{ij}$.

The quality of feasible solutions can be assessed comparing its value with the corresponding VRP best known solution value z^*, using the *GAP* measure:

$$GAP(s) = 100\left(\frac{f(s)}{z^*} - 1\right). \tag{8}$$

Overtime constraints ensure that feasible solutions satisfy $f(s) \leqslant mT$, which for problems with time horizon T_1, implies that *GAP* will be upper-bounded by a value near 5%. Analogously, for problems with T_2, the *GAP* upper bound will be close to 10%.

Three articles provide computational experience regarding these benchmark problems [7,8,11]. In all cases, feasible solutions to some problem instances could not be found. The *GAP* is not an adequate quality measure for these cases, and special measures must be defined. The total overtime (3) measures the time spent beyond the planning horizon. The longest tour ratio

$$LTR(s) = \frac{\max_{k \in R(s)} \sum_{r \in R_k} t_r}{T} \tag{9}$$

considers the duration of the vehicle which arrives latest in terms of the planning horizon. The penalized cost measures the routing cost that results from increasing c_{ij} by a factor of δ after the time horizon has elapsed:

$$PC(s) = f(s) + (\delta - 1)\, O(s). \tag{10}$$

The algorithm was coded in C++. All tests were done on a 1.8 GHz AMD Athlon XP 2200+ processor with 480 MBytes of RAM under Windows XP.

Table 1
Running times for configurations that found feasible solutions for all tuning instances

AMP^{iter}	TS^{iter}	M^{size}	p	Min.	Max.
300	500	250	15	34	354
300	750	250	15	48	533
300	750	150	30	109	880
300	750	250	30	108	888

4.2. Parameter tuning

As in most meta-heuristics, there are a number of parameters which may modify the algorithm's behavior, namely:

- The number of AMP iterations: AMP^{iter}.
- The maximum number of routes in the memory: M^{size}.
- The number of initial solutions: I.
- The number of TS iterations: TS^{iter}.
- The neighborhood size: p.
- The interval in which tabu tenures are drawn: $[\theta^{\min}, \theta^{\max}]$.
- The bounds of α and β: $\alpha^{\min}$, $\beta^{\min}$, $\alpha^{\max}$ and $\beta^{\max}$.

In order to reduce the tuning effort, some parameters that are believed to be less influencing on the whole algorithm's performance were arbitrarily fixed. Those values are $\alpha^{\min}=\beta^{\min}=1$ and $\alpha^{\max}=\beta^{\max}=100,000$ for the bounds, $\theta^{\min}=5$ and $\theta^{\max}=10$ for the tabu tenure interval and $I=20$ for the number of initial solutions.

For each of the remaining parameters a set of candidate values was proposed, from which the final value was to be selected: $AMP^{\text{iter}} \in \{100, 200, 300\}$, $M^{\text{size}} \in \{150, 250\}$, $TS^{\text{iter}} \in \{250, 500, 750\}$ and $p \in \{15, 30\}$. To select the final values, the AMP procedure was ran over a subset of six benchmark problems, with every possible candidate value combination. The test problems selected for tuning were CMT-1, CMT-4, CMT-5, CMT-5, CMT-12 and F-11, using time horizon T_1 and $m = 2, 8, 9, 7, 5$ and 2 respectively.

For each run we recorded the running time and whether a feasible solution was found or not. Only the parameter configurations which led to a feasible solution for the six instances were further analyzed. For each, the minimum and maximum running times (in seconds) over the tuning problems are given in Table 1.

The selected parameter values were those which reported lowest computation times and obtained a feasible solution for every tuning instance: $AMP^{\text{iter}}=300$, $TS^{\text{iter}}=500$, $M^{\text{size}}=250$ and $p=15$. During the parameter tuning, it has been observed that for all problems, except CMT-5 (with $m=9$), a feasible solution was found within the first 200 AMP iterations. So, every 100 iterations the best solution found so far will be checked for feasibility, and the algorithm will keep running only if the solution is not feasible.

Table 2
Detailed computational results for problems CMT-1, CMT-2 and CMT-3

Problem	m	T	Feas.	*GAP*				Time	PS	BM	TLG
				Best	Avg.	Worst	St.dev.				
CMT-1	1	551	5	0.0	0.0	0.0	0.0	10	✓	✓	✓
$n = 50$	2	275	5	1.6	1.7	2.2	0.3	10	×	✓	✓
$z^* = 524.61$	3	184	0	—	—	—	—	33	×	×	×
	4	138	0	—	—	—	—	30	×	×	×
	1	577	5	0.0	0.0	0.0	0.0	10	✓	✓	✓
	2	289	5	1.0	1.1	1.6	0.3	11	✓	✓	✓
	3	192	5	5.4	5.5	6.0	0.3	12	✓	✓	×
	4	144	5	4.1	4.2	4.3	0.1	11	×	✓	✓
CMT-2	1	877	5	0.0	0.2	0.3	0.1	21	✓	✓	✓
$n = 75$	2	439	5	0.2	0.5	1.0	0.4	21	✓	✓	✓
$z^* = 835.26$	3	292	5	0.2	0.5	1.3	0.5	21	✓	✓	✓
	4	219	5	0.1	0.7	1.7	0.7	22	✓	✓	✓
	5	175	5	1.2	1.6	2.1	0.4	22	✓	✓	✓
	6	**146**	1	2.6	2.6	2.6	—	71	×	×	×
	7	125	0	—	—	—	—	66	×	×	×
	1	919	5	0.1	0.7	1.3	0.5	21	✓	✓	✓
	2	459	5	0.1	0.5	1.0	0.4	21	✓	✓	✓
	3	306	5	0.1	0.4	0.8	0.3	22	✓	✓	✓
	4	230	5	0.1	0.2	0.6	0.2	22	✓	✓	✓
	5	184	5	0.1	0.9	1.3	0.5	23	✓	✓	✓
	6	153	5	2.5	3.4	4.8	0.9	23	✓	✓	✓
	7	131	5	3.6	4.2	4.7	0.4	23	×	✓	×
CMT-3	1	867	5	0.0	0.4	0.6	0.2	24	✓	✓	✓
$n = 100$	2	434	5	0.4	0.5	1.0	0.2	23	✓	✓	✓
$z^* = 826.14$	3	289	5	0.5	0.7	0.9	0.2	25	✓	✓	✓
	4	217	5	0.4	0.8	1.2	0.3	23	✓	✓	✓
	5	173	5	2.6	3.2	4.1	0.6	25	×	✓	×
	6	**145**	5	1.2	2.0	3.0	0.7	77	×	×	×
	1	909	5	0.4	0.5	0.7	0.2	24	✓	✓	✓
	2	454	5	0.3	0.4	0.7	0.2	24	✓	✓	✓
	3	303	5	0.2	0.4	0.4	0.1	23	✓	✓	✓
	4	227	5	0.0	0.3	0.5	0.2	25	✓	✓	✓
	5	182	5	0.8	1.0	1.9	0.5	26	✓	✓	✓
	6	151	5	1.1	1.7	1.9	0.4	26	✓	✓	✓

4.3. Results

The algorithm was run over the whole benchmark set using the selected parameter configuration. For each problem instance, five independent runs of the AMP were performed. Detailed results are presented

in Tables. 2–4. The meaning of each column is the following:

- Problem: the base VRP problem.
- m and T: the number of vehicles and time horizon. For each base problem, instances with T_1 are listed first. Bold entries indicate that a new feasible solution was found by the AMP.

Table 3
Detailed computational results for problems CMT-4, CMT-5 and CMT-11

Problem	m	T	Feas.	*GAP*				Time	PS	BM	TLG
				Best	Avg.	Worst	St.dev.				
CMT-4	1	1080	5	0.5	0.9	1.3	0.4	51	✓	✓	✓
$n = 150$	2	540	5	0.8	1.7	3.5	1.1	53	✓	✓	✓
$z^* = 1028.42$	3	360	5	0.7	1.3	1.8	0.4	52	✓	✓	✓
	4	270	5	0.8	1.6	2.5	0.9	53	×	✓	✓
	5	216	5	0.4	1.6	2.8	0.9	54	✓	✓	✓
	6	180	5	2.0	3.2	4.5	0.9	55	×	✓	✓
	7	154	0	—	—	—	—	160	×	×	×
	8	**135**	3	3.0	3.6	4.3	0.7	53	×	×	×
	1	1131	5	1.0	1.3	1.6	0.2	52	✓	✓	✓
	2	566	5	0.9	1.5	2.1	0.5	55	✓	✓	✓
	3	377	5	0.5	1.1	1.6	0.4	53	✓	✓	✓
	4	283	5	1.0	1.7	2.5	0.5	54	✓	✓	✓
	5	226	5	0.8	1.5	2.3	0.6	52	✓	✓	✓
	6	189	5	0.4	1.4	2.0	0.7	54	✓	✓	✓
	7	162	5	2.6	3.3	3.9	0.6	57	×	✓	✓
	8	141	5	3.5	4.4	5.6	0.8	55	✓	✓	×
CMT-5	1	1356	5	1.9	2.8	3.4	0.6	101	✓	✓	✓
$n = 199$	2	678	5	2.0	3.1	3.9	0.8	101	✓	✓	✓
$z^* = 1291.44$	3	452	5	1.6	2.1	2.8	0.4	102	✓	✓	✓
	4	339	5	2.5	3.0	3.2	0.3	100	✓	✓	✓
	5	271	5	2.5	3.1	4.1	0.6	104	×	✓	✓
	6	226	5	3.1	3.5	4.0	0.4	105	✓	✓	✓
	7	194	5	3.6	3.8	4.1	0.2	105	×	✓	✓
	8	170	5	2.8	3.5	4.2	0.6	103	×	✓	✓
	9	151	4	3.4	3.7	4.0	0.3	104	×	×	✓
	10	**136**	1	4.1	4.1	4.1	—	327	×	×	×
	1	1421	5	2.1	2.7	3.5	0.7	103	✓	✓	✓
	2	710	5	1.8	2.4	2.9	0.5	105	✓	✓	✓
	3	474	5	1.6	2.3	2.6	0.4	105	✓	✓	✓
	4	355	5	1.3	2.4	3.6	1.0	105	✓	✓	✓
	5	284	5	2.0	2.7	3.3	0.5	106	✓	✓	✓
	6	237	5	1.5	2.7	4.1	1.0	108	✓	✓	✓
	7	203	5	2.3	3.2	3.7	0.6	108	✓	✓	✓
	8	178	5	1.8	2.0	2.3	0.2	104	✓	✓	✓
	9	158	5	3.0	3.4	4.2	0.5	106	✓	✓	✓
	10	142	5	4.1	4.6	5.0	0.4	108	×	✓	✓

Table 3 (*continued*)

Problem	m	T	Feas.	*GAP*				Time	PS	BM	TLG
				Best	Avg.	Worst	St.dev.				
CMT-11	1	1094	5	0.1	2.5	3.2	1.3	19	✓	✓	✓
$n = 120$	2	547	5	3.0	3.1	3.2	0.1	23	×	✓	✓
$z^* = 1042.11$	3	365	5	0.2	2.7	4.5	2.1	22	×	✓	✓
	4	**274**	2	4.2	4.3	4.4	0.1	62	×	×	×
	5	219	5	0.1	0.8	1.3	0.4	23	×	✓	✓
	1	1146	5	0.2	2.5	3.1	1.3	22	✓	✓	✓
	2	573	5	1.2	2.6	3.1	0.8	21	✓	✓	✓
	3	382	5	0.1	0.8	3.3	1.4	22	✓	✓	✓
	4	287	5	0.1	0.2	0.4	0.1	23	×	✓	✓
	5	229	4	0.1	0.3	0.6	0.2	25	✓	✓	✓

- Feas.: the number of runs that produced a feasible solution.
- *GAP*: the measure defined in (8), taken only over feasible solutions. The best, average and worst value considering the five runs are reported. The standard deviation is given for instances in which at least two runs produced a feasible solution.
- Time: the running time of the best run[1] (in seconds).
- PS, BM, TLG: indicates whether a feasible solution was found (✓) or not (×) by Petch and Salhi [11], Brandão and Mercer [9] and Taillard et al. [7], respectively.

The AMP found feasible solutions to 98 out of 104 problem instances, including 6 which were unsolved by the previous proposals [7,9,11]. A feasible solution was found for each of the 52 instances with T_2, as it is the case with the procedure proposed by Brandão and Mercer [9]. Taillard et al. [7] and Petch and Salhi [11] report 48 and 46 feasible solutions, respectively. Instances with T_1 are more difficult, as they have a shorter time horizon. The proposed AMP found 46 feasible solutions, Brandão and Mercer [9] found 37, Taillard et al. [7] 38 and Petch and Salhi [11] 30.

Regarding the robustness of the algorithm, in most instances the *GAP* standard deviation was very small. Moreover, only in seven instances happened that some runs produced a feasible solution while others did not.

Summarized results considering only the best run for each instance are given in Table 5. Each row refers to the instances obtained from the same base VRP problem. For each time horizon, the following results are reported:

- Feas./Total: the number of instances for which a feasible solution was found and the total number of problem instances.
- *GAP*: the average *GAP* (8) considering only the problems in which a feasible solution was found.
- Time: The average running time in seconds.
- The last row shows the total of Feas./Total column and the average *GAP*.

[1] The best run for a problem instance is the one which produced the least cost feasible solution. If the five runs produced infeasible solutions, the *LTR* measure is used to determine the best run.

Table 4
Detailed computational results for problems CMT-12, F-11 and F-12

Problem	m	T	Feas.	*GAP*				Time	PS	BM	TLG
				Best	Avg.	Worst	St.dev.				
CMT-12	1	861	5	0.0	0.1	0.2	0.1	21	✓	✓	✓
$n = 100$	2	430	5	0.0	<0.1	0.2	0.1	22	✓	✓	✓
$z^* = 819.56$	3	287	5	0.0	<0.1	<0.1	<0.1	22	✓	✓	✓
	4	215	5	0.0	0.0	0.0	0.0	22	✓	×	✓
	5	172	5	3.1	3.3	3.4	0.1	22	✓	×	×
	6	143	0	—	—	—	—	71	×	×	×
	1	902	5	0.0	<0.1	< 0.1	< 0.1	22	✓	✓	✓
	2	451	5	0.0	<0.1	<0.1	<0.1	21	✓	✓	✓
	3	301	5	0.0	<0.1	<0.1	<0.1	23	✓	✓	✓
	4	225	5	0.0	<0.1	<0.1	<0.1	23	✓	✓	✓
	5	180	5	0.6	0.6	0.6	0.0	23	✓	✓	✓
	6	150	5	0.4	0.5	0.7	0.1	23	✓	✓	✓
F-11	1	254	5	0.0	0.3	0.7	0.3	8	✓	✓	✓
$n = 71$	**2**	**127**	4	4.2	4.3	4.3	0.1	10	×	×	×
$z^* = 241.97$	3	85	0	—	—	—	—	25	×	×	×
	1	266	5	0.0	0.4	0.7	0.3	8	✓	✓	✓
	2	133	5	0.0	0.8	1.8	0.8	9	✓	✓	✓
	3	89	5	5.4	7.0	8.2	1.3	9	✓	✓	×
F-12	1	1221	5	0.5	0.6	0.9	0.2	30	✓	✓	✓
$n = 134$	2	611	5	0.7	0.9	1.1	0.2	27	✓	✓	✓
$z^* = 1162.96$	3	407	5	0.3	0.5	0.7	0.2	31	✓	✓	✓
	1	1279	5	0.8	1.0	1.3	0.2	29	✓	✓	✓
	2	640	5	0.7	0.9	1.0	0.1	28	✓	✓	✓
	3	426	5	0.4	0.8	1.0	0.3	31	✓	✓	✓

The algorithm finds high quality solutions. Feasible solutions found are, on average, within 1.27% of the VRP best known solution. As the hardest to achieve for the benchmark problem set is feasibility and as every feasible solution is close to the VRP best known solution (see Section 4.1), other authors do not report the cost of the feasible solutions found. Thus, no comparison of this aspect can be made.

There are six instances, all of which use T_1 as time horizon, that could not be solved by the AMP nor by the previous approaches [7,9,11]. In Table 6 infeasible solutions are compared in terms of their *LTR* values. Each row corresponds to an instance for which every run produced an infeasible solution. For each instance, only the best of the five runs is reported. Average values and the difference between each algorithm and the AMP are given in the last two rows. The algorithm obtains the best results in all but one instance. The average *LTR* value over the six infeasible instances is significantly smaller than the one

Table 5
Summarized results considering the best run for each problem instance

Problem	T_1			T_2		
	Feas./Total	*GAP*	Time	Feas./Total	*GAP*	Time
CMT-1	2/4	0.80	21	4/4	2.62	11
CMT-2	6/7	0.72	35	7/7	0.95	22
CMT-3	6/6	0.87	33	6/6	0.47	25
CMT-4	7/8	1.16	66	8/8	1.33	54
CMT-5	10/10	2.74	125	10/10	2.15	106
CMT-11	5/5	1.52	30	5/5	0.34	23
CMT-12	5/6	0.63	30	6/6	0.18	23
F-11	2/3	2.10	14	3/3	1.80	9
F-12	3/3	0.48	29	3/3	0.64	29
	46/52	1.37	—	52/52	1.19	—

Table 6
Longest tour ratio (*LTR*) comparison for infeasible solutions

	Problem	m	AMP	Brandão and Mercer [9]	Taillard et al. [7]	Petch and Salhi [11]
1	CMT-1	3	**1.024**	1.041	1.115	1.026
2	CMT-1	4	**1.027**	**1.027**	**1.027**	1.085
3	CMT-2	7	**1.009**	1.088	1.073	1.064
4	CMT-4	7	**1.002**	1.071	1.033	1.072
5	CMT-12	6	**1.014**	1.072	1.064	1.029
6	F-11	3	1.020	**1.011**	1.075	1.020
Average			**1.016**	1.052	1.065	1.049
Average difference (%)			—	−3.31	−4.49	−3.12

obtained by other authors. These results suggest that the proposed algorithm is highly accurate in finding routes which fit the time horizon.

Table 7 compares the cost of the infeasible solutions with those obtained by Brandão and Mercer [9] and Taillard et al. [7] (Petch and Salhi [11] do not report costs for the infeasible solutions). For each instance, the cost (1), the total overtime (3) and the penalized cost (10) using $\delta = 2$ are given. The last row shows, for each measure, the average difference between each algorithm and the AMP.

The values must be analyzed carefully. As it was pointed in Section 2.4, fitting the time horizon and minimizing the routing costs may be opposed objectives. In the general case, it is unlikely that both can be simultaneously attained. On one hand, if high overtime values are admitted, solutions with smaller cost (but highly infeasible) may be found. On the other hand, if the admitted overtime is small, some customers may have to be relocated in order to shorten some of the longest routes, but increasing the total cost.

Table 7
Total overtime (O) and penalized cost (PC) comparison for infeasible solutions

	AMP			Brandão and Mercer [9]			Taillard et al. [7]		
	Cost	O	PC	Cost	O	PC	Cost	O	PC
1	558.82	**7.29**	**573.40**	556.34	9.70	575.73	**533.00**	23.24	579.48
2	547.10	**8.49**	**564.07**	547.10	**8.49**	**564.07**	**546.29**	9.49	565.27
3	873.40	**1.86**	**877.12**	870.07	13.25	896.57	**843.60**	17.35	878.29
4	1072.82	**0.32**	**1073.46**	1063.95	19.34	1102.63	**1042.39**	15.89	1074.16
5	852.99	3.81	860.61	**819.56**	**3.19**	**825.94**	**819.56**	12.96	845.48
6	256.85	1.85	260.55	254.40	**1.54**	257.47	**244.60**	6.36	**257.31**
	Average difference (%)			1.12	−28.26	0.03	3.42	−67.99	0.26

Table 8
Running time comparison

Problem	n	Instances	AMP	TLG	BM	PS
CMT-1	50	16	16	300	150	108
F-11	71	6	13	1560	150	258
CMT-2	75	14	29	420	300	330
CMT-3	100	12	27	1440	600	828
CMT-12	100	12	27	1380	600	120
CMT-11	120	10	28	2700	1500	2430
F-12	134	6	31	4500	4800	810
CMT-4	150	16	68	3060	1500	984
CMT-5	199	20	125	3960	3750	2454

The routing costs given by Taillard et al. [7] are, in all cases, the smallest. However, the total overtime values obtained by our algorithm are lower. This is not surprising, since Taillard et al. [7] assign routes to vehicles in the last phase of their algorithm, while our AMP modifies the assignment at every iteration. While Taillard et al. [7] obtain low cost routes that are hard to pack in working days, our AMP produces solutions with slightly higher costs (3.42%) but which are dramatically better in terms of feasibility (67.99%). Overall, in terms of PC values, solutions obtained by the AMP algorithm are slightly outperformed by Taillard et al. [7] and Brandão and Mercer [9] when $\delta = 2$. However, the good compromise between infeasibility and cost obtained by the AMP is captured when using higher values of δ.

Table 8 provides information about running times in seconds. For each base problem, the average running time (considering the five runs for each instance) is given under the AMP column. The last three columns give the average running times reported by Taillard et al. [7] (Silicion Graphics Indigo, 100 MHz), Brandão and Mercer [9] (HP Vectra XU Pentium Pro, 200 MHz) and Petch and Salhi [11] (Ultra Enterprise 450, 300 MHz), respectively.

On average, the AMP ran 51, 30 and 22 times faster than the algorithms by Taillard et al. [7], Brandão and Mercer [9] and Petch and Salhi [11], respectively. However, a fair comparison of the approaches in

terms of running times is extremely difficult to attain, given the big difference between the AMP running environment and the others.

5. Conclusion and future work

An algorithm based on the Adaptive Memory Programming principle has been proposed to solve the Vehicle Routing Problem (VRP) with Multiple Trips, an important extension of the classic VRP.

The algorithm was ran over a set of benchmark problems and the solutions obtained were compared with those reported for three previously proposed algorithms [7,9,11]. Our algorithm obtained more feasible solutions than the previous approaches. Further analysis shows that a good compromise between routing cost and overtime violation is achieved for highly constrained instances.

The proposed algorithm can be extended to solve more realistic problems. For instance, minor changes in the way that routes are assigned to vehicles may allow to handle heterogeneous fleet problems. It must be noticed, however, that incorporating time windows is a harder problem since the assignment of routes to vehicles cannot be solved via a Bin Packing Problem.

Alternatives aimed at achieving diversification should be tested. Regarding the population management, it would be interesting to implement a mechanism to favor the diversity of the routes stored in the adaptive memory, because it may help to avoid premature converge.

Finally, it would be useful to design an algorithm which efficiently solved the model presented in Section 2.3 over a restricted set of routes. Such an algorithm could be used in a final post-optimization step of the AMP, as it is done in petal algorithms [29].

Acknowledgements

The authors are grateful to the anonymous referees, whose comments helped to improve the presentation of this paper and the quality of the reported results.

Appendix A. New feasible solutions

Feasible solutions to problems for which all previous approaches were infeasible are presented next.

New feasible solution to CMT-2 with $m = 6$ and $T_1 = 146$.

Veh.	Duration	Node sequence
1	135.4808	(0, 72, 10, 31, 55, 25, 50, 18, 24, 49, 0)
2	115.8165	(0, 47, 36, 69, 71, 60, 70, 20, 37, 15, 57, 13, 0)
2	28.9126	(0, 17, 40, 0)
3	115.2378	(0, 73, 1, 22, 64, 42, 43, 41, 56, 23, 0)
3	29.0774	(0, 67, 46, 34, 4, 75, 0)
4	95.3259	(0, 8, 54, 19, 59, 14, 53, 7, 0)
4	49.5767	(0, 51, 16, 63, 33, 6, 0)

5	81.8075	(0, 58, 38, 65, 66, 11, 35, 0)
5	63.2687	(0, 3, 44, 32, 9, 39, 12, 26, 0)
6	78.1909	(0, 68, 2, 62, 28, 61, 21, 74, 0)

New feasible solution to CMT-3 with $m = 6$ and $T_1 = 145$.

Veh.	Duration	Node sequence
1	139.7459	(0, 70, 30, 20, 66, 65, 71, 35, 34, 78, 33, 81, 9, 51, 1, 69, 0)
2	139.2430	(0, 89, 60, 83, 45, 8, 46, 36, 49, 64, 63, 90, 32, 10, 31, 27, 0)
3	139.0635	(0, 50, 3, 79, 29, 24, 54, 55, 25, 39, 67, 23, 56, 4, 0)
4	137.0156	(0, 2, 57, 15, 43, 42, 14, 44, 38, 86, 16, 61, 17, 84, 5, 0)
5	93.2598	(0, 18, 82, 48, 47, 19, 11, 62, 88, 7, 52, 0)
5	51.4576	(0, 28, 76, 77, 68, 80, 12, 26, 0)
6	81.8540	(0, 53, 58, 40, 21, 73, 72, 74, 75, 22, 41, 87, 97, 13, 0)

New feasible solution to CMT-4 with $m = 8$ and $T_1 = 135$.

Veh.	Duration	Node sequence
1	134.5364	(0, 110, 4, 56, 75, 23, 67, 39, 139, 25, 55, 24, 29, 121, 80, 109, 0)
2	134.3165	(0, 1, 122, 51, 120, 9, 103, 66, 71, 65, 136, 35, 135, 34, 78, 0)
3	130.0318	(0, 60, 84, 17, 113, 86, 140, 38, 44, 119, 14, 142, 42, 43, 15, ... , 57, 144, 0)
4	128.6711	(0, 146, 7, 19, 107, 11, 64, 49, 143, 36, 47, 124, 46, 8, 114, 18, 0)
5	89.9579	(0, 132, 69, 101, 70, 30, 20, 128, 131, 32, 90, 63, 126, ... , 108, 10, 31, 0)
5	44.9832	(0, 112, 94, 95, 117, 97, 87, 137, 13, 0)
6	70.5012	(0, 6, 59, 93, 85, 61, 16, 141, 91, 100, 37, 98, 92, 0)
6	62.9857	(0, 28, 76, 116, 77, 3, 79, 129, 81, 33, 102, 50, 111, 0)
7	70.2185	(0, 27, 127, 88, 148, 62, 123, 48, 82, 106, 52, 0)
7	64.2336	(0, 105, 26, 149, 130, 54, 134, 150, 68, 12, 138, 0)
8	68.5059	(0, 89, 118, 83, 45, 125, 5, 104, 99, 96, 147, 0)

New feasible solution to CMT-5 with $m = 10$ and $T_1 = 136$.

Veh.	Duration	Node sequence
1	135.9972	(0, 189, 10, 126, 63, 181, 64, 49, 143, 36, 47, 168, 124, 46, 0)
2	134.8522	(0, 84, 17, 113, 86, 140, 38, 44, 119, 192, 14, 142, 42, 172, 43, ... , 15, 57, 0)
3	133.5093	(0, 66, 65, 136, 35, 135, 164, 34, 78, 169, 121, 29, 24, 163, 134, 0)
4	95.7858	(0, 139, 187, 39, 67, 170, 25, 55, 165, 130, 54, 195, 26, 0)
4	36.8802	(0, 146, 52, 89, 147, 6, 183, 156, 0)
5	87.4173	(0, 132, 69, 162, 108, 90, 32, 131, 160, 128, 188, 20, 30, 70)

5	48.3207	(0, 96, 99, 104, 93, 85, 98, 92, 151, 95, 94, 112, 0)
6	87.2733	(0, 176, 1, 122, 51, 103, 161, 71, 9, 120, 81, 33, 0)
6	48.6725	(0, 28, 184, 116, 68, 80, 150, 177, 109, 12, 138, 154, 0)
7	83.5545	(0, 105, 180, 197, 56, 186, 23, 75, 133, 22, 41, 145, 115, 0)
7	52.1047	(0, 13, 117, 97, 87, 144, 178, 2, 137, 58, 152, 0)
8	81.7143	(0, 148, 62, 159, 11, 175, 107, 19, 123, 48, 82, 0)
8	53.8775	(0, 27, 167, 127, 190, 31, 88, 182, 7, 194, 106, 153, 0)
9	72.1859	(0, 166, 5, 173, 61, 16, 141, 191, 91, 193, 100, 37, 59, 0)
9	57.0604	(0, 111, 50, 102, 157, 185, 79, 129, 3, 158, 77, 196, 76, 0)
10	68.7432	(0, 53, 40, 21, 73, 171, 74, 72, 198, 110, 155, 4, 179, 149, 0)

New feasible solution to CMT-11 with $m = 4$ and $T_1 = 274$.

Veh.	Duration	Node sequence
1	233.6144	(0, 57, 40, 43, 45, 48, 51, 50, 49, 47, 46, 44, 41, 42, 39, 38, 37, 0)
1	34.1421	(0, 82, 81, 119, 120, 0)
2	217.8109	(0, 52, 54, 59, 65, 61, 62, 64, 66, 63, 60, 58, 56, 55, 53, 68, 0)
2	55.1095	(0, 87, 92, 89, 91, 90, 114, 18, 118, 108, 83, 113, 117, 84, 112, ... , 85, 86, 111, 88, 0)
3	208.7067	(0, 17, 16, 19, 25, 22, 24, 27, 33, 30, 31, 34, 36, 35, 29, 32, 28, ... , 26, 23, 20, 21, 109, 0)
3	62.8379	(0, 105, 107, 102, 101, 99, 100, 116, 98, 110, 115, 97, 94, 93, ... , 96, 95, 0)
4	138.7362	(0, 67, 69, 70, 71, 74, 72, 75, 78, 80, 79, 77, 76, 73, 103, 104, ... , 106, 0)

New feasible solution to CMT-2 with $m = 2$ and $T_1 = 127$.

Veh.	Duration	Node sequence
1	97.0179	(0, 9, 7, 4, 8, 3, 5, 10, 6, 71, 12, 16, 17, 13, 1, 15, 2, 19, 62, ... , 64, 65, 63, 59, 57, 41, 55, 54, 0)
1	28.3711	(0, 35, 18, 11, 14, 31, 0)
2	68.4026	(0, 20, 29, 23, 26, 24, 25, 49, 51, 70, 50, 47, 48, 52, 45, 53, ... , 46, 44, 43, 42, 27, 28, 22, 21, 30, 0)

References

[1] Toth P, Vigo D. The vehicle routing problem. SIAM Monographs on Discrete Mathematics and Applications, 2002.
[2] Rochat Y, Taillard E. Probabilistic diversification and intensification in local search for vehicle routing. Journal of Heuristics 1995;1:147–67.
[3] Lenstra J, Rinnooy Kan A. Complexity of vehicle routing and scheduling problems. Networks 1981;11:221–7.
[4] Fleischmann B. The vehicle routing problem with multiple use of vehicles. Working paper, Fachbereich Wirtschaftswissenschaften, Universität Hamburg, 1990.

[5] Clarke G, Wright W. Scheduling of vehicles from a central depot to a number of delivery points. Operations Research 1964;12:568–81.
[6] Martello S, Toth P. Knapsack problems. Chichester: Wiley; 1990.
[7] Taillard E, Laporte G, Gendreau M. The vehicle routing problem with multiple use of vehicles. Journal of the Operational Research Society 1996;47:1065–70.
[8] Brandão J, Mercer A. A tabu search algorithm for the multi-trip vehicle routing and scheduling problem. European Journal of Operational Research 1997;100:180–91.
[9] Brandão J, Mercer A. The multi-trip vehicle routing problem. Journal of the Operational Research Society 1998;49: 799–805.
[10] Gendreau M, Hertz A, Laporte G. New insertion and post optimization procedures for the travelling salesman problem. Operations Research 1992;40:1086–94.
[11] Petch R, Salhi S. A multi-phase constructive heuristic for the vehicle routing problem with multiple trips. Discrete Applied Mathematics 2004;133:69–92.
[12] Yellow P. A computational modification to the savings method of vehicle scheduling. Operational Research Quarterly 1970;21:281–3.
[13] Balinsky M, Quandt R. On an integer program for a delivery problem. Operations Research 1964;12:300–4.
[14] Christofides N, Mingozzi A, Toth P. The vehicle routing problem. In: Combinatorial optimization. Chichester: Wiley; 1979. p. 315–38.
[15] Glover F. Heuristics for integer programming using surrogate constraints. Decision Sciences 1977;8:156–66.
[16] Tarantilis C, Kiranoudis C. BoneRoute: an adaptive memory-based method for effective fleet management. Annals of Operations Research 2002;114:227–41.
[17] Tarantilis C. Solving the vehicle routing problem with adaptive memory programming methodology. Computers and Operations Research 2005;32:2309–27.
[18] Gendreau M, Laporte G, Musaraganyi C, Taillard E. A tabu search heuristic for the heterogeneous fleet vehicle routing problem. Computers and Operations Research 1999;26:1153–73.
[19] Crevier B, Cordeau J-F, Taillard E. The multi-depot vehicle routing problem with inter-depot routes. Technical Report CRT-2004-10, Centre de recherche sur les transports, Montreal, 2004.
[20] Hvattum L, Løkketangen A, Glover F. Adaptive memory search for boolean optimization problems. Discrete Applied Mathematics 2004;142:99–109.
[21] Bozkaya B, Erkut E, Laporte G. A tabu search heuristic and adaptive memory procedure for political districting. European Journal of Operational Research 2003;144:12–26.
[22] Tang H, Miller-Hooks E. A TABU search heuristic for the team orienteering problem. Computers and Operations Research 2005;32:1379–407.
[23] Ahmadi S, Osman I. Greedy random adaptive memory programming search for the capacitated clustering problem. European Journal of Operational Research 2005;162:30–44.
[24] Taillard E, Gambardella L, Potvin MGJ-Y. Adaptive memory programming: a unified view of metaheuristics. European Journal of Operational Research 2001;135:1–16.
[25] Gillett B, Miller L. A heuristic algorithm for the vehicle-dispatch problem. Operations Research 1974;22:340–9.
[26] Taillard E. Parallel iterative search methods for vehicle routing problems. Networks 1993;23:661–76.
[27] Gendreau M, Hertz A, Laporte G. A tabu search heuristic for the vehicle routing problem. Management Science 1994;40:1276–90.
[28] Fisher M. Optimal solution of vehicle routing problems using minimum k-trees. Operations Research 1994;42:626–42.
[29] Foster B, Ryan D. An integer programming approach to the vehicle scheduling problem. Operational Research Quarterly 1976;27:367–84.